

A Proposal for Enhancing Agile Requirements Engineering with Prototyping and Enriched User Stories

Nader Keshk

Department of Computer Sciences,
Faculty of Computers and Artificial
Intelligence, Cairo University, Cairo,
Egypt
naderpro83@gmail.com

Mohammad El-Ramly

Department of Computer Sciences,
Faculty of Computers and Artificial
Intelligence, Cairo University, Cairo,
Egypt
m.elramly@fci-cu.edu.eg

Akram Salah

Department of Computer Sciences,
Faculty of Computers and Artificial
Intelligence, Cairo University, Cairo,
Egypt
akram.salah@fci-cu.edu.eg

ABSTRACT

Agile software development is the most developed and widely used family of software development processes. Agile Requirement Engineering (ARE) faces many challenges, such as managing with very little documentation and specifications, ensuring that the development team understands the requirements, and understanding all client needs. In agile methodologies, user stories are the primary means for capturing requirements. But most of the time, user stories are not enough to describe the requirements to the development team to the required level of details. Moreover, sometimes, the client does not have a clear vision of the system features from the beginning. S/he discovers them during or even after delivery, leading to an increase in the amount of rework. Hence, some researchers have suggested solutions to these challenges by using prototypes to support the user stories in explaining the requirements. Others suggested using enriched user stories to get more details about how to implement the requirements. In this paper, we discuss the challenges facing ARE, the role that each of prototyping and enriched user stories can play in improving ARE, and the limitations of each one if used alone. We propose a roadmap to improve ARE by a hybrid process that merges prototyping and enriched user stories along with extra validation steps.

CCS CONCEPTS

• **Software and its engineering** → Software creation and management; Designing software; Requirements analysis.

KEYWORDS

Requirements Engineering, Agile, Prototyping, User Story

ACM Reference Format:

Nader Keshk, Mohammad El-Ramly, and Akram Salah. 2022. A Proposal for Enhancing Agile Requirements Engineering with Prototyping and Enriched User Stories. In *Federated Africa and Middle East Conference on Software Engineering (FAMECSE '22)*, June 07, 08, 2022, Cairo-Kampala, Egypt. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3531056.3542773>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
FAMECSE '22, June 07, 08, 2022, Cairo-Kampala, Egypt
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9663-9/22/06...\$15.00
<https://doi.org/10.1145/3531056.3542773>

1 INTRODUCTION

An agile software development methodology is a set of practices that support continuous iteration of the development, testing, and delivery cycle during the software development project. Demands and solutions evolve through self-organizing and cross-functional teams and their clients [1]. While agility is more of a mindset, several methodologies were developed following this approach, like Extreme Programming (XP), Kanban, Scrum, and others [2], with Scrum being the most popular agile methodology [3].

The integration of software with all aspects of life, the rapid change of business needs, and the competition force organizations to move from traditional heavyweight software development methodologies to flexible lightweight agile ones. But agile development methods were primarily designed for small co-located teams, with high access to the client in a less formal setting. Organizations attempted to scale up the agile development methods to fit their globalization and distribution context [4].

According to Brooks, software complexity is intrinsic to the very nature of software itself [5]. Agile methodologies face challenges when moving to complex, large, distributed, and/or critical projects. The challenges are not related to any specific agile method but are general to all agile software development methods [6]. In this work, we address one particular challenge that faces agile methodologies when applied to complex projects: the challenge of adequately capturing and conveying requirements to the developers, i.e., requirements engineering (RE).

The central concept in agile methods is reducing documentation and increasing collaboration between team members and stakeholders, with a focus on producing working software at the end of each cycle or sprint. This is challenging to implement in reality in many projects due to the unavailability of the client for quick communications. Moreover, there are several common well-studied challenges for ARE: 1. Requirements change [7, 9, 13–15], 2. Managing with very little documentation and specifications [6, 7, 9, 11, 12, 15], 3. Requirements sometimes are vague [8, 9, 13], 4. Missing requirements [9, 14], 5. validation and verification [9, 27], 6. Ensuring all the development team and clients share the same understanding of requirements [8], 7. Conflicting requirements [7–9] and, 8. The client's availability. [7–9, 16]

In agile methods, user stories are the primary means to capture requirements. They are written in a lightweight user-driven format and are intended to open a dialogue between the client and the development team. In practice, they are insufficient to properly convey requirements to the developers in large, complex, and distributed projects, with ambiguous and sometimes conflicting requirements

and not-so-accessible clients. For example, the developers need to know how screens will look like, which screens will be affected by the user story, which data tables are affected, and what data pieces will be transferred to/from the tables.

In this research, we address ARE challenges identified by previous studies. We propose a comprehensive enhanced ARE process that uses both prototyping and enriched user stories, along with extra validation steps to tackle these challenges, reduce the amount of misunderstanding and rework and ensure maximum quality and clarity of requirements within agile projects.

2 AGILE REQUIREMENTS ENGINEERING CHALLENGES

Challenges facing agile requirements engineering (ARE) are well studied in the literature, which shows identical challenges. The sequel presents ARE challenges as found by the key researches on the topic. Table 1 summarizes the findings of these researches.

Inayat et al. applied a systematic literature review on papers published between 2002 and June 2013 and identified 21 papers that discuss agile requirements engineering. They concluded that the practice of ARE poses 8 challenges: 1. Minimal documentation, 2. Client availability, 3. Budget and schedule estimation, 4. Inappropriate architecture, 5. Neglecting non-functional requirements, 6. Client inability and agreement (Domain knowledge is incomplete, and client groups are divided on how to proceed), 7. Contractual limitations and requirements volatility, and 8. Requirements change and change evaluation [7].

Gizzatullina conducted a study of the client communication problem in ARE using two systematic literature reviews and semi-structured interviews. Results show many problems: 1. Product owners (PO) need to simplify their language to make clients understand what they explain, 2. Developers do not participate in requirements writing. Hence, they are far from understanding the user's needs, 3. Clients disagree with the product vision. Clients and the team may have different interpretations of the same things, 4. On the development team's side, they do not want to know the business part, 5. On the client's side, his representatives may not give time to collaborate with the group [8].

Batra and Bhatnagar conducted a study on 80+ experts in the field for ARE to capture challenges in requirements engineering that affect the overall software development cycle. The results show that the most critical challenges are: 1. Weak effort estimation, 2. Inappropriate Architecture, and 3. Minimal Documentation [6].

Aqsa et al. conducted a literature review for the studies published from 1998 to 2020 about the challenges being faced during ARE. Their findings are as follows: 1. Lack of documentation, 2. Missing, ambiguous, and conflicting requirements, 3. Less direct communication with stakeholders, 4. Less preliminary planning and focus on initial team involvement, 5. Less experienced and skilled team, 6. The inability of clients to tell user stories, 7. Lack of prototyping, 8. Tacit knowledge, 9. Changing requirements, 10. Requirement prioritization and 11. Neglecting non-functional requirements. In addition, there are RE challenges related to large projects, e.g., 1. Missing high-level requirement management, 2. Weak requirement in the start, 3. Weak requirement prioritization, 4. Validation and verification, and 5. Communication [9].

Table 1: Agile Requirements Engineering Challenges

ID	ARE Challenge	Resources
RC1	Minimal documentation	[6, 7, 9, 11, 12, 15]
RC2	Ambiguous requirements	[8–10, 13]
RC3	Missing requirements	[9, 14]
RC4	Conflicting requirements	[7–9]
RC5	Client availability	[7–9, 16]
RC6	Requirements change	[7, 9, 13–15]
RC7	Missing high-level requirement	[9, 25]
RC8	Inappropriate architecture	[7]
RC9	Neglecting non-functional requirements	[7, 9]
RC10	Budget and schedule estimation	[6, 7]
RC11	Less preliminary planning	[9, 13]
RC12	Less experienced and skilled team	[9, 26]
RC13	Tacit knowledge	[9, 12]
RC14	Requirement prioritization	[9]
RC15	Weak/shallow requirement in the start	[9]
RC16	Validation and verification	[9, 27]
RC17	Communication	[7–9]

Amna et al did a literature review to investigate and propose remedies for ambiguity in user stories. The vagueness of needs articulated as user stories, inconsistencies between user stories, the inadequacy of user stories in capturing requirements, and duplication of requirements in many user stories were all noted in 165 papers in the ambiguity class. They concluded: "we need a complete solution for dealing with the user stories ambiguity" [10].

3 USING PROTOTYPING IN ARE

A few studies tackled ARE challenges by integrating prototyping with the ARE process. Prototypes can be mockups developed by design tools like Adobe XD (can be of high or low fidelity) or prototypes developed programmatically (and costly) with HTML, CSS, JS, etc. We summarize the relevant works below.

Käpyaho and Kauppinen conducted a case study on a large agile project to explore how prototyping can solve agile requirements engineering challenges [17]. They programmed prototypes using HTML, CSS, and JavaScript. They measured their results by comparing the case study results to the findings from the literature. Their findings suggest that prototyping is an effective technique in solving some of the agile requirements engineering challenges. For example, prototypes can act as concrete plans that can be relied on when discussing changes. On the other hand, there are some cons to their approach: 1. They used an unrealistic technique to gather the initial requirements by conducting meetings with significant participation including most development teams. 2. They created interactive prototypes by combining HTML, CSS, and JavaScript. However, utilizing this technique at this early stage is not the best solution for responding to changes in requirements because changing prototypes would take time. It is better to use mockups as

prototypes in representing requirements as they are flexible and faster to modify than codes.

Tanvir et al. merged prototyping with the agile software development methodology. They introduced the concept of reusing components from agile development by producing reusable prototyping. As a result, rather than designing a component from scratch, existing components would be reused. [18]. Despite their findings, this is not a reliable method as it would be more accurate to use a reusable component from a completed project rather than a prototype repository.

Beni Suranto conducted an empirical study on a software project developing a Student Marking System (SMS). Using a one-week iteration agile process, software engineers always decide which functionalities to represent next using prototypes. The software engineers used a combination of low-fidelity and high-fidelity prototypes during the requirements engineering process. The results show that both low-fidelity and high-fidelity prototypes have their advantages and disadvantages. However, high-fidelity prototypes allow stakeholders to test the system and validate requirements more thoroughly. They mimic the final system's behavior, allowing stakeholders to test the system's primary prospective functionalities. As a result of this benefit, stakeholders will make suggestions and/or propose new requirements. In comparison to low-fidelity prototypes, high-fidelity prototypes allow software engineers to deliver a more full and engaging system and allow stakeholders to explore and test the functionality in details [19]. In our approach, prototyping is done with high-fidelity design mockups.

4 USING ENHANCED USER STORIES IN ARE

Other researches aimed to solve agile requirements challenges by improving the requirements definition in agile methodology, particularly by enhancing the primary unit of ARE, i.e., user stories. Table 2. Shows some proposed enhancements.

Natalia Juristo et al. used user stories to incorporate the functional usability suggestions [20]. They've identified three ways for including usability in users: 1. New stories have been included to represent usability requirements, 2. Tasks in current user stories can be added or changed to represent the behaviors of usability. This task might be as easy or as complex as necessary, and 3. Acceptance criteria are added or modified to include usability needs. Their final results still need evaluation and empirical validation. But early feedback suggests that adding usability features to user stories was simple and useful [21].

Lopes et al. experimented with 18 agile Brazilian agile developers to write up user stories for a specific software application. They were given a set of User eXperience Design (UxD) artifacts to help them with this mission. They were also asked to identify which UxD artifacts they had used and to explain why they chose one over the other. The findings suggest that employing UxD artifacts during the writing of user stories can provide at least two benefits: (a) improved requirement descriptions for software quality, and (b) increased developer knowledge of UX concerns. As a result, by creating a shared vocabulary for both developers and designers, this strategy of employing UxD artifacts may improve communication between them [22].

Table 2: Research on User Story Enrichment / Enhancement

User Story Enrichment / Enhancement	Resources
Adding Usability Requirements	[21]
Using the "user experience design artifacts" to enhance the user story writing	[22]
Replacing user story with RSD including conceptual modeling, mockups modeling, and acceptance criteria	[23]
Enhanced user story template to automate object models	[24]

Juliana Medeiros et al. conducted two case studies to evaluate the Requirements Specification for Developers (RSD), including conceptual modeling, mockups modeling, and acceptance criteria. This approach proposes developing Software Requirements Specifications (SRS) that provide information closer to development needs. The results showed that RSD approach met the developer's expectations and produced a more objective SRS suitable for coding activities. However, SRS is not intended to be used as a mechanism for requirements validation with the client. Also, there is a need for prioritizing and categorizing the acceptance criteria. Moreover, there is a need for a tool for creating, searching and tracing the artifacts [23].

Adil et al. proposed an Object-oriented Analysis and Design Approach for RE (OOADARE) to create an enhanced readable template of user stories to automate the production of object models. Despite this enhancement, the user story needs more clarifications to be closer to the development team's needs [24].

5 A HYBRID APPROACH FOR ENHANCING AGILE REQUIREMENTS ENGINEERING

Based on the above discussion and the limitations of using prototyping and enriched stories alone, we propose a hybrid approach to an enhanced ARE process that tackles the challenges mentioned in Table 1. This process uses a set of practices and activities together that help improve ARE. The process consists of ascending steps, each one builds on the previous one as below.

Step 1: Gathering the high-level requirements and building a business process model. In this step, high-level requirements are collected, recorded in a suitable format, and reviewed to ensure that nothing is missing. This gives a complete picture of the project's size and a general view of the project modules and features. Business process modeling is done as a flowchart that shows a precise sequence of business activities and data flows required to complete a process. This step is critical as it gives all team members, especially the solution architect, a reliable picture of the whole project to build a stable architecture for the project's development. This model provides the PO with a road map to collect the details of requirements. Also, it guarantees building the software incrementally without reworks caused by overlooking the dependencies of related features, which leads to delays and affects the project schedule, and weakens the software structure.

Step 2: Selecting a feature to implement. Based on the client's priority, the PO selects a feature with no dependency on

other unfinished features for eliciting detailed functional, non-functional, and design requirements. Also, the UI/UX designer attends the requirements gathering sessions to collect design requirements. UI/UX designer engagement in gathering detailed requirements sessions is essential to cover the design requirements at early-stage and helps in building the prototypes considering the client's needs in the design since the beginning of the project. This reduces the amount of UI rework.

Step 3: Building a prototype. Prototyping is a very efficient technique to gather requirements. It is very effective in sharing client needs with the development team. But we propose the following additional practices from our experience to achieve the maximum possible benefit of using prototyping. 1. Using high-fidelity mockups as prototypes in representing requirements. This is flexible and faster than building prototypes with codes. 2. Using a professional tool for developing the prototype. This accelerates productivity. 3. Generating an evolutionary prototype that evolves into a final product. This helps in accelerating the development by reusing the prototype rather than starting from scratch.

Step 4: Validating and verifying prototypes with the stakeholders. This helps avoiding missing any requirements and exposes any conflicting requirements. Also, it ensures that PO and UI/UX designers share the same understanding with stakeholders.

Step 5: Validating prototypes with the technical team. This helps avoiding any technical issues that may arise in the future and reduces the amount of rework. It also ensures the feasibility of the prototype from the development point of view.

Step 6: Writing an enriched user story. After validating prototypes, the PO creates a detailed user story. An enhanced user story template is used which explains how to implement the feature in greater details to ensure that the team accurately understands the user story. This template includes: user story statement, actors (in case of multiple users), priority, precondition, post-condition, scenarios, output, data dictionary, and enhanced acceptance criteria (It specifies not just business rules, but also any other form of constraint that the system requires, such as non-functional and validation constraints.) and the prototype. This overcomes the lack of documentation and ensures that stakeholders and developers have the same understanding. It also ensures that the non-functional requirements are not neglected.

Step 7: User story review. The client reviews the user story details with the prototype and gives his final approval. The client's needs are validated at this step. It serves as a second evaluation for the prototype, double-checking the user story details to ensure that all client requirements are met and none is missing.

Step 8: Backlog update. PO adds the finished user story to the backlog and repeats steps 2 to 8 till finishing all requirements.

The development team can start a sprint if the backlog has enough completed user stories. PO and UI/UX designer work in parallel with the development team and must precede it by one sprint at least. This approach can be used with XP, Scrum, Kanban, etc.

While change is still welcome and the spirit of agility is still maintained, using this workflow reduces the impact of change and the amount of rework, by putting extra effort upfront in detailing and validating requirements, instead of putting it later in rework.

Table 3: Expected Impact on Resolving ARE Challenges

Proposed Enhanced ARE Step / Artifact	IDs of Solved Agile Requirements Challenges
High-level requirements / Business Process Model Building Prototypes	RC2, RC3, RC7, RC8, RC11, RC14, RC15 RC1, RC2, RC3, RC4, RC5, RC6, RC10, RC12, RC13, RC15, RC17
Validating and verifying prototypes with the stakeholders	RC3, RC4, RC6, RC10, RC16
Validating prototypes with the technical team	RC4, RC6, RC10, RC16
Writing / Reviewing enriched user stories	RC1, RC2, RC3, RC5, RC6, RC8, RC9, RC10, RC12, RC13, RC15, RC17

This does not delay development, because it is done in parallel to it. Table 3 shows how our ARE process resolves ARE Challenges.

6 CONCLUSION AND FUTURE WORK

This paper presented the challenges that ARE faces which lead to misunderstanding and increase rework. We presented two approaches used by researchers to reduce these two issues, namely prototyping, and enriched user stories. Then we developed a proposal for merging prototyping and enriched user stories, along with additional practices, to be deployed in a new ARE process that tackles the challenges in traditional ARE. Our enhanced ARE process combines enhanced procedures, an integrated workflow, prototyping, and enriched user stories. We developed an enriched user story template and an enhanced ARE workflow to implement our proposal. Currently, we are using them in a large-scale project for empirical evaluation. Also, we are planning to survey experts in the field on how they value and evaluate our approach.

REFERENCES

- [1] Guru 99, Agile Methodology, [Online], Accessed 02/04/2022, <https://www.guru99.com/agile-scrum-extreme-testing.html>
- [2] A.Qumer, and B.Henderson-Sellers. 2008. An evaluation of the degree of agility in six agile methods and its applicability for method engineering. *Information and Software Technology*, 50(4), pp.280-295.
- [3] Samar Alsaqqa, Samer Sawalha, and Heba Abdel-Nabi. 2020. Agile Software Development: Methodologies and Trends. *International Journal of Interactive Mobile Technologies*, 14(11). <https://doi.org/10.3991/ijim.v14i11.13269>
- [4] Maria Paasivaara and Casper Lassenius. 2011, September. Scaling scrum in a large distributed project. In 2011 International Symposium on Empirical Software Engineering and Measurement (pp. 363-367). IEEE.
- [5] Frederick P. Brooks. 1995. *The Mythical Man-Month*, Anniversary Edition. Reading, Massachusetts: Addison Wesley.
- [6] Mona Batra and Archana Bhatnagar. 2019. A Research Study on Critical Challenges in Agile Requirements Engineering. *International Research Journal of Engineering and Technology (IRJET)*, Volume: 06, 2019, pp.1214-1219.
- [7] Irum Inayat, Siti Salwah Salim, Sabrina Marczak, Maya Daneva, and Shahaboddin Shamshirband., 2015. A systematic literature review on agile requirements engineering practices and challenges. *Computers in Human Behavior*, 51, pp.915-929. <https://doi.org/10.1016/j.chb.2014.10.046>
- [8] Ilyuza Gizzatullina. 2019, August. Empirical study of customer communication problem in agile requirements engineering. In Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (pp. 1262-1264).
- [9] Aqsa Rasheed, Bushra Zafar, Tehmina Shehryar, Naila Aiman Aslam, Muhammad Sajid, Nouman Ali, Saadat Hanif Dar and Samina Khalid. 2021. Requirement

- Engineering Challenges in Agile Software Development. *Mathematical Problems in Engineering*, 2021. <https://doi.org/10.1155/2021/6696695>
- [10] Anis R. Amna and Geert Poels. 2022. Ambiguity in User Stories: A Systematic Literature Review. *Information and Software Technology*, p.106824.
 - [11] Muhammad Amir, Khalid Khan, Adnan Khan and M.N.A. Khan. 2013. An appraisal of agile software development process. *International Journal of Advanced Science & Technology*, 58(56), p.20.
 - [12] N.Ganesh and S.Thangasamy.2011. Issues identified in the software process due to barriers found during eliciting requirements on agile software projects: Insights from India. *International J. of Computer Applications*, 16(5), pp.7-12.
 - [13] Hassan Hajjdiab and Al Shaima Taleb. 2011. Adopting agile software development: issues and challenges. *International Journal of Managing Value and Supply Chains (IJMVSC)*, 2(3), pp.1-10.
 - [14] Waleed Helmy, Amr Kamel and Osman Hegazy. 2012. Requirements engineering methodology in agile environment. *International Journal of Computer Science Issues (IJCSI)*, 9(5), p.293.
 - [15] Yanda Sebege and Ernest Mnkandl. 2017. Exploring issues in agile requirements engineering in the South African software industry. *The Electronic J. of Information Systems in Developing Countries*, 81(1), pp.1-18.
 - [16] Sehrish Alam, S. Asim, Asim, Shahid Bhatti and Amr Jadi. 2017. Impact and challenges of requirement engineering in agile methodologies: A systematic review. *Int. J. of Advanced Computer Science and App.*, 8(4), pp.411-420.
 - [17] Marja Käpyaho and Marjo Kauppinen., 2015, August. Agile requirements engineering with prototyping: A case study. In 2015 IEEE 23rd International Requirements Engineering Conference (RE) (pp. 334-343). IEEE.
 - [18] Sara Tanvir, Mamoona Safdar, Hanny Tufail and Usman Qamar. 2017. Merging prototyping with agile software development methodology. *Int. Conf. on Engineering, Computing & IT (ICECIT 2017)* (pp. 50-54).
 - [19] Beni Suranto. 2015, August. Software prototypes: Enhancing the quality of requirements engineering process. In 2015 Int. Symposium on Technology Management and Emerging Technologies (ISTMET) (pp. 148-153). IEEE.
 - [20] Natalia Juristo, Ana Moreno and Maria-Isabel Sanchez-Segura. 2007. Guidelines for eliciting usability functionalities. *IEEE Transactions on Software Engineering*, 33(11), pp.744-758. DOI: 10.1109/TSE.2007.70741
 - [21] Ana M. Moreno and Agustín Yagüe. 2012, May. Agile user stories enriched with usability. In *Int. Conf. on Agile Software Development* (pp. 168-176).
 - [22] Larissa A. Lopes, Eduardo G. Pinheiro, Tiago S. da Silva and Luciana A. M. Zaina. 2018, May. Using UxD artifacts to support the writing of user stories: Findings of an empirical study with agile developers. In *Proceedings of the 19th Int. Conference on Agile Software Development: Companion* (pp. 1-4).
 - [23] Juliana Medeiros, Alexandre Vasconcelos, Carla Silva and Miguel Goulão. 2020. Requirements specification for developers in agile projects: Evaluation by two industrial case studies. *Info. and Software Technology*, 117, p.106194.
 - [24] Adil Zeaaraoui, Zineb Bougroun, Mohammed G. Belkasmí and Toumi Bouchentouf. 2013, August. User stories template for object-oriented applications. In *Third International Conference on Innovative Computing Technology (INTECH 2013)* (pp. 407-410). IEEE.
 - [25] Kim Dikert, Maria Paasivaara. and Casper Lassenius. 2016. Challenges and success factors for large-scale agile transformations: A systematic literature review. *Journal of Systems and Software*, 119, pp.87-108.
 - [26] D. Méndez Fernández, S. Wagner, M. Kalinowski, M. Felderer, P. Mafra, A. Vetrò, T. Conte, M.-T. Christiansson, D. Greer, C. Lassenius and T. Männistö. 2017. Naming the pain in requirements engineering. *Empirical software engineering*, 22(5), pp.2298-2338. <https://doi.org/10.1007/s10664-016-9451-7>
 - [27] Elizabeth Bjarnason, Krzysztof Wnuk and Björn Regnell. 2011, July. A case study on benefits and side-effects of agile practices in large-scale requirements engineering. In *Proc. of 1st Workshop on Agile Requirements Eng.* (pp. 1-5).